

Tarea Programada 2

Evaluador de expresiones matemáticas

Felipe Arce Castillo

José David Serrano Robles

Mariana Torres Valverde

Escuela de Ingeniería en Computación, Tecnológico de Costa Rica, San José

Abstract – Este programa es una calculadora de operaciones básicas tales como la suma, resta, multiplicación y división. Permite al usuario ingresar 10 variables las cuales ingresará dentro de una expresión para luego asignarles a cada una un valor numérico. Fue implementado en ensamblador para AA32.

Keywords – Ensamblador, AA32, calculadora

I. INTRODUCCIÓN:

El documento presente muestra brevemente una descripción general del funcionamiento del programa a través de un diagrama de bloques y un diagrama de flujo, además de una explicación de los distintos algoritmos y procedimientos implementados.

Esta calculadora es una aplicación de consola que permite al usuario ingresar una expresión matemática con distintas operaciones básicas como suma, resta, multiplicación y división. Esta puede contener o no variables las cuales serán asignadas por el propio usuario.

La implementación en ensamblador de arquitectura ARM permite tener un mayor control de la computadora y ofrecer un

rendimiento significativo para realizar las operaciones necesarias. La aplicación fue desarrollada en un Raspberry Pi 4 que cuenta con la arquitectura AA32.

Para llevar a cabo esta tarea se implementó un algoritmo que resuelve operaciones matemáticas tomando en cuenta la prioridad de estas y el uso de paréntesis, este mismo se detalla en la explicación general del diseño.

II. OBJETIVOS

1. **Objetivo General:** Generar una solución de bajo nivel al problema de resolver operaciones aritméticas.

Objetivos Específicos:

- Mostrar las ventajas del uso de ensamblador para procesos de bajo nivel
- Analizar el algoritmo de resolución de operaciones aritméticas avanzadas.

III. EXPLICACIÓN DE DISEÑO Y ARQUITECTURA

La calculadora primero presenta una serie de casos distintos que se pueden presentar a la hora de evaluar un carácter como por ejemplo si la cantidad de caracteres excede el límite (1024) o si hay un paréntesis sin cerrar.

Después de validar si la expresión es correcta se valida si las variables fueron introducidas correctamente y las reemplaza por sus valores numéricos. Posteriormente busca los paréntesis y resuelve las operaciones en ellos.

Luego se realizan todas las operaciones aritméticas con los valores resultantes de los paréntesis, si es una multiplicación o división se opera de una vez, si es una suma o una resta se guarda el índice, en este último caso se invoca una subrutina 'searchPriority' para buscar operaciones de mayor prioridad, si no se encuentran se realiza la suma o la resta.

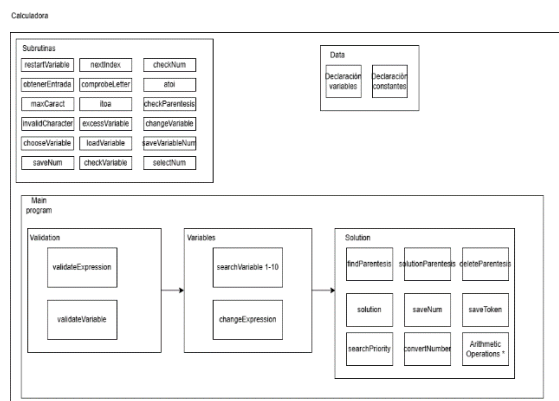


Figura 1 Diagrama de Bloques

Subrutinas:

Este bloque contiene algunas subrutinas del programa:

restartVariable: Esta subrutina recorre una variable y reemplaza todos sus bytes con ceros.

obtenerEntrada: Permite obtener la entrada de teclado por consola del usuario.

maxCaract: Si la cantidad de caracteres excede el límite (1024) se llama esta subrutina, se imprime un mensaje de error y termina el programa.

invalidCharacter: Si el carácter no es una operación o un número se llama esta

subrutina, se imprime el mensaje de error y se termina el programa.

chooseVariable: carga una variable en un registro para poder cargar el valor en esta.

saveNum: Se guarda carácter por carácter el número para operarlo posteriormente, hay una subrutina similar para el segundo número.

nextIndex: Al validar la expresión se incrementa el índice cuando se encuentra un carácter válido para validar el siguiente.

comprobeLetter: Esta subrutina comprueba si el carácter actual es una letra, caso contrario manda un mensaje de error.

Itoa: Siglas de Integer to Ascii, obtiene carácter por carácter y realiza la resta ascii para obtener el valor correspondiente de cada dígito para después multiplicar por 10 y sumar el carácter.

excessVariable: Si el usuario usa más de 10 variables se imprime un mensaje de error y se termina el programa.

loadVariable: Guarda la incógnita ingresada por el usuario en un espacio de memoria para ser reemplazada por el valor numérico más adelante.

checkVariable: Busca la variable a reemplazar y llama a la subrutina para cambiar su valor.

checkNum: Reocorre la expresión para validar si está bien escrita, revisa que no hayan caracteres incorrectos, en caso de que lo haya imprime un mensaje de error y termina el programa.

comprobeNum:

atoi: Siglas de Ascii to Integer. Realiza la descomposición numérica y extrae dígito a dígito usando el residuo para formar el número con los caracteres en ascii.

checkParentesis: Busca los paréntesis de la expresión para operar su contenido de primero, si encuentra un paréntesis sin cerrar envía un mensaje de error y finaliza el programa..

changeVariable: En una variable aparte copia la expresión matemática, esta subrutina se encarga de copiar el valor numérico de la variable en esta nueva expresión.

saveVariableNum: Busca el número para operar.

selectNum: Selecciona el número a almacenar en la variable.

Data:

Declaración variables: Se declaran todas las reservas de memoria que serán usadas para almacenar datos.

Declaración constantes: Tanto los mensajes de error para imprimir como las constantes numéricas están definidas en esta sección.

Programa Principal:

Validation:

- validateExpression: primero se valida que la expresión no tenga caracteres inválidos ni paréntesis sin cerrar.
- validateVariable: Asigna a los registros las variables a guardar.

Variables:

- searchVariable 1 - 10: Se almacenan los números de las incógnitas en distintos espacios de memoria.
- changeExpression: Se sobrescribe la expresión matemática ingresada por el usuario, esta vez

reemplazando las variables por su respectivo valor numérico

Solution:

- parenthesis: Se resuelven los paréntesis y se valida si estos están escritos correctamente.
- Solution: Se buscan 2 números a operar y se almacenan en registros junto con su índice en la variable de la expresión.
- saveToken: Se guarda la operación aritmética a realizar en una variable.
- searchPriority: Se busca una operación con mayor prioridad, si no se encuentra se realiza la operación del inicio de la expresión.

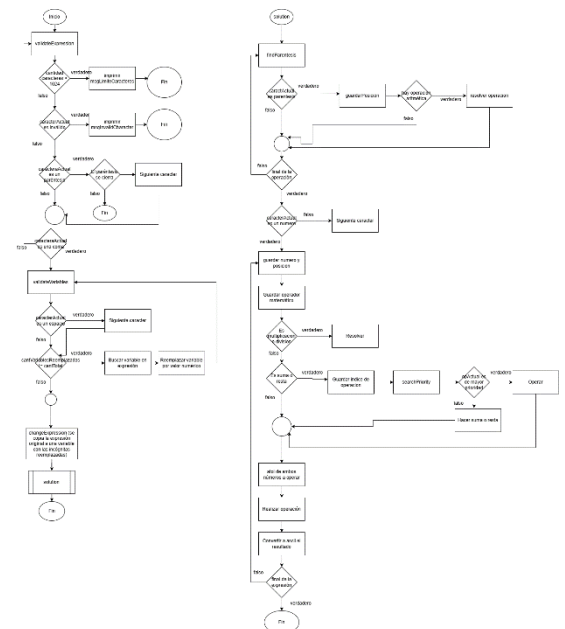


Figura 2 Diagrama de flujo

IV. CONCLUSIONES:

- Ensamblador facilita la modificación de caracteres al poder acceder a cualquier parte de un texto, sin embargo, se necesita tener un buen manejo de los índices de cada variable
- Es esencial en ensamblador indicar el formato de entrada al usuario, al ser un lenguaje tan complejo es complicado e ineficiente hacer un programa completamente robusto, es necesaria la comunicación con el usuario y la explicación correcta de la forma de utilizar el programa
- Es necesario tener bien claro el flujo del programa ya que, a la hora de implementar cambios o corregir errores, el orden de ejecución de las distintas subrutinas y algoritmos pueden generar confusión. Para esto, el proceso de diseño y creación los diagramas son útiles.

para validar la eficiencia y confiabilidad de estos.

V. RECOMENDACIONES:

- Para este tipo de proyectos es útil tener alguna forma de mostrar en pantalla los valores que está trabajando el programa para facilitar el manejo de errores y el buen funcionamiento del programa
- Es especialmente útil en ensamblador utilizar separadores de texto como comas o espacios para facilitar la obtención y manejo de datos.
- Antes de diseñar un programa que ejecute algoritmos matemáticos, estos se pueden plantear con pseudocódigo y realizar pruebas antes de programar

